Fundamentals Security Application Web □□□□□□□□□□□

دليل تعليمي شامل — منهجية الحل بدون إعطاء الإجابات النهائية

المحتويات

مقدمة الدليل **3**

فلسفة هذا الدليل 3

كيفية استخدام هذا الدليل 3

الروم الأول: Application An Walking **4**

نظرة عامة على الروم 4

Application An Walking — 1 Task (الإعداد) 4

Website The Exploring — 2 Task (استكشاف الموقع) 4

Source Page The Viewing — 3 Task (عرض الكود المصدري) 5

Inspector Tools: Developer — 4 Task 6

Debugger Tools: Developer — 5 Task 7

Network Tools: Developer — 6 Task 8

Storage Tools: Developer — 7 Task 8

Conclusion — 8 Task (خاتمة الروم) 9

الروم الثاني: Discovery Content **10**

نظرة عامة على الروم 10

Introduction — 1 Task (الإعداد) 10

Files Common Discovery: Manual — 2 Task (الملفات الشائعة) 10

Stack Framework & Headers Discovery: Manual — 3 Task (الهيدرز وبصمة الإطار) 11

Tools Web & Engines Search OSINT: — 4 Task (محركات البحث والأدوات) 12

Archives & Repositories OSINT: — 5 Task (الأرشيف والمستودعات) 12

Fundamentals Gobuster Discovery: Automated — 6 Task (أساسيات Gobuster) 13

Hosts Virtual & Subdomains Discovery: Automated — 7 Task (النطاقات الفرعية والمضيفون الافتراضيون) 14

Conclusion — 8 Task (خاتمة الروم) 16

الروم الثالث: Stacks Web Modern **17**

نظرة عامة على الروم 17

Introduction — 1 Task (الإعداد) 17

Express Stack MERN — 2 Task (Prototype + Pollution) 17

Middleware Next.js / React — 3 Task (Bypass) 19

SQL Django — 4 Task (Injection عبر order_by) 20

Apache LAMP — 5 Task (Traversal Path 2.4.49 □ RCE) 22

Task 6 — Automation (الفحص الآلي بـ Nikto) 23
Task 7 — Conclusion (خاتمة الروم) 24

الروم الرابع: Attacks Server Web - I **25**
نظرة عامة على الروم 25
Task 1 — Introduction (الإعداد) 25
Task 2 — Servers Web Identfiying (تحديد هوية الخوادم) 25
Task 3 — Server HTTP Python (لا يوجد تحكم وصول إطلاقاً) 26
Task 4 — Apache2 ،mod_status) Directory ،Listing والملفات المنسية) 27
Task 5 — Node.js :(Express) التسريبات في وضع التطوير 28
Task 6 — autoindex Nginx) و stub_status) 29
Task 7 — Common Misconfigurations Across Servers (الأنماط المشتركة بين كل الخوادم) 31
Task 8 — Conclusion (خاتمة الروم) 32

ملخص سريع للمراجعة النهائية (Cheat Sheet) **33**
الروم 1 — Application An Walking 33
الروم 2 — Discovery Content 33
الروم 3 — Stacks Web Modern 33
الروم 4 — I - Attacks Server Web 33

قائمة مراجعة المهارات **34**

مقدمة الدليل

هذا الدليل تعليمي بالكامل، وهو يغطي مديول **Fundamentals Security Application Web** من منصة TryHackMe، ويشمل الرومات الأربعة التالية بالترتيب:

1. **Application An Walking** — الفحص اليدوي لتطبيقات الويب باستخدام أدوات المتصفح فقط.
2. **Discovery Content** — اكتشاف المحتوى المخفي بثلاث طرق: يدوي، OSINT، وآلي.
3. **Stacks Web Modern** — بصمة الـ Stacks الحديثة (MERN، Next.js، Django، LAMP) واستغلال ثغرات حقيقية (CVEs) فيها.
4. **I - Attacks Server Web** — التعرف على أخطاء الإعداد الشائعة في أربعة خوادم ويب مختلفة (Apache، Nginx، Server، Node.js/Express، HTTP Python).

فلسفة هذا الدليل

الهدف هنا ليس تزويدك بقائمة إجابات جاهزة (Flags) لنسخها ولصقها. هذا أسلوب يفقدك أهم شيء في اختبار الاختراق: **بناء المنهجية الذهنية**. لذلك، لكل سؤال في كل تاسك، سنتبع نفس البنية:

- **ما الفكرة التي يختبرها السؤال؟** — شرح المفهوم الأمني أو التقني وراء السؤال.
- **أين تجد المعلومة؟** — تحديد المكان بالضبط (هيدر، HTTP، كود مصدري، مخرجات أداة...).
- **كيف تصل للإجابة؟** — الأمر أو الخطوة العملية المحددة.
- **كيف تتحقق من صحتها؟** — طريقة للتأكد قبل إرسال الإجابة في المنصة.

القيم النهائية (الـ Flags بصيغة `THM{...}` أو كلمات المرور المكتشفة) **متروكة لك لاستخراجها بنفسك** من بيئة المعمل الخاصة بك، لأن هذه القيم غالباً عشوائية ومختلفة من نسخة إلى أخرى من الآلة، والأهم أن استخراجها بنفسك هو ما يُكسبك المهارة فعلياً.

كيفية استخدام هذا الدليل

اقرأ شرح المفهوم أولاً، نفّذ الأمر بنفسك على بيئتك، راقب المخرجات، وقارنها بما هو موضح هنا من ناحية الشكل والبنية (وليس القيمة الحرفية). إذا علقت، ارجع لقسم "التلميحات الإضافية" في نهاية كل تاسك.

الروم الأول: Application An Walking

نظرة عامة على الروم

هذا الروم يعلّمك كيف تراجع تطبيق ويب يدوياً باستخدام أدوات المتصفح المدمجة فقط (بدون أي أداة فحص آلية). الفكرة الأساسية: الأدوات الآلية كثيراً ما تفوّت معلومات وثغرات موجودة في التعليقات البرمجية، الروابط المخفية، سلوك الـ JavaScript، وطلبات الشبكة. العين المدربة أقوى من أي سكانر في هذه المرحلة.

ما ستتدرب عليه: - قراءة الكود المصدري القابل للقراءة البشرية (Page Source) - فحص عناصر الصفحة وتعديلها حياً عبر الـ Inspector - التحكم في تنفيذ الـ JavaScript عبر الـ Debugger وتقنية الـ Breakpoints - تتبع طلبات الشبكة (Network) التي تصدرها الصفحة. - فحص البيانات المخزنة محلياً في المتصفح (Storage: Local Cookies، Storage، إلخ).

Application An Walking — 1 Task (الإعداد)

هذا التاسك إعدادي بحت: تشغيل الـ AttackBox والـ Machine Lab والتأكد من الوصول لموقع Support IT Acme عبر Firefox. لا يوجد سؤال تقني هنا، فقط تأكيد.

ملاحظة منهجية: في كل رومات TryHackMe، أول تاسك يكون غالباً إعداد بيئة فقط — خصص وقته للتأكد أن كل الخدمات تعمل قبل الانتقال لأي تحليل فني، لأن أي خطأ هنا (IP لم يتحدّث بعد، أو الآلة لم تكتمل إقلاعها) سيُفسد كل الخطوات اللاحقة.

Website The Exploring — 2 Task (استكشاف الموقع)

المفهوم

قبل أي اختبار اختراق فعلي، يجب أن تبني **خريطة ذهنية** لتطبيق الويب: ما هي الصفحات الموجودة؟ ما الذي يقبل مدخلات من المستخدم؟ أين توجد نقاط تسجيل الدخول؟ هذه الخطوة تسمى أحياناً "Application Mapping"، وهي الأساس الذي تُبنى عليه كل الاختبارات اللاحقة.

المنهجية

تصفح الموقع يدوياً صفحة صفحة، ولاحظ: - أي صفحة تحتوي نموذج إدخال ؟(Form) - أي رابط يحتوي باراميتر في الـ URL (مثل `?id=1`)؟ هذه نقاط محتملة لاختبار حقن (Injection). - أي ميزة تتطلب مصادقة (تسجيل دخول) أم متاحة للجميع؟

النتيجة المتوقعة هي جدول يشبه ما هو موضح في التاسك: يحتوي على اسم الميزة، المسار (Endpoint)، ووصف موجز لوظيفتها.

السؤال

ما هو الـ endpoint الخاص بإنشاء تذاكر جديدة؟

أين تجد المعلومة؟ تصفح قسم Customers بعد تسجيل الدخول، وابحث عن زر "Create Ticket" في لوحة التحكم (Dashboard)، ثم لاحظ الرابط (URL) الذي ينتقل إليه المتصفح.

كيف تتحقق؟ راقب شريط العنوان في المتصفح عند الضغط على الزر، أو افحص الرابط في الكود المصدري داخل الزر نفسه (`href` أو `action` الخاص بالفورم).

استنتج الإجابة بنفسك من الملاحظة المباشرة أعلاه.

Task 3 — Viewing The Source Page (عرض الكود المصدري)

المفهوم

الكود المصدري Page) (Source هو ما يُرسله الخادم فعلياً للمتصفح (HTML+CSS+JS) قبل أي تعديل يقوم به الـ JavaScript لاحقاً. يختلف هذا عن الـ DOM الحي الذي تراه في الـ Inspector (سنغطيه في التاسك التالي). مراجعة هذا الكود تكشف:

- **تعليقات المطورين (Comments)**: نصوص بين `<!--` و `-->` لا تظهر للمستخدم لكنها موجودة في الكود، وغالباً ما تحتوي ملاحظات تطويرية حساسة (مسارات تجريبية، أسباب تأجيل ميزة، إلخ).
- **روابط مخفية**: عناصر `<a>` موجودة في الكود لكنها غير ظاهرة بصرياً (مثلاً عبر CSS يخفيها، أو لا يوجد نص واضح لها).
- **مسارات الملفات الثابتة (Static Assets)**: روابط صور/CSS/JS تكشف عن بنية المجلدات على الخادم، وأحياناً تكشف وجود مجلد مفتوح للفهرسة (Directory Listing).
- **بصمة الـ Framework**: تعليقات في أسفل الصفحة تذكر اسم وإصدار الإطار المستخدم.

المنهجية خطوة بخطوة

1. **كليك يمين ← View Page Source**، أو اكتب `view-source:` قبل الرابط في شريط العنوان.
2. ابحث (Ctrl+F) عن `<!--` لإيجاد كل التعليقات في الصفحة.
3. ابحث عن وسوم `<a href=` لحصر كل الروابط، وقارنها بما هو ظاهر فعلياً في الواجهة — أي رابط موجود في الكود لكن غير ظاهر في الصفحة هو "رابط مخفي" يستحق الفحص.
4. لاحظ مسار الملفات الثابتة (مثل `/assets/`)، وجرّب فتح هذا المسار مباشرة في المتصفح كمجلد (بدون اسم ملف) لترى إن كان الخادم يعرض فهرس الملفات (Directory Listing) بدلاً من رسالة خطأ 403/404.
5. ابحث عن تعليق في أسفل الصفحة يذكر اسم الإطار (Framework) وإصداره ورابط موقعه الرسمي.

لماذا يهم هذا؟

معرفة اسم وإصدار الـ Framework تفتح الباب أمام البحث عن ثغرات معروفة (CVEs) له، كما أن معرفة بنية صفحة الإدارة الافتراضية للإطار قد تقودك لصفحة تسجيل دخول لم تكن لتجدها بالتخمين.

الأسئلة

ما هي القيمة المستخرجة من تعليق الـ HTML؟

أين تجد المعلومة؟ في أعلى الكود المصدري للصفحة الرئيسية، ستجد تعليقاً يشير إلى صفحة بديلة قيد التطوير (رابط يبدأ بمسار مختلف عن الصفحة الرئيسية). افتح هذا المسار مباشرة.

ما هي القيمة المستخرجة من الرابط السري؟

أين تجد المعلومة؟ ابحث داخل الكود المصدري عن كلمة تبدأ بـ `secr` ضمن وسم `<a href=`. هذا الرابط موجود في الكود لكنه غير مرتبط بأي نص ظاهر أو مخفي بصرياً عبر CSS. افتحه مباشرة في المتصفح.

ما هي قيمة فلاج الفهرسة المفتوحة للمجلد (Directory Listing)؟

أين تجد المعلومة؟ لاحظ مسار تحميل ملفات الصور/CSS/JS (مثل `/assets/`) من الكود المصدري، ثم افتح هذا المسار كمجلد مباشرة في المتصفح. إذا ظهرت لك قائمة بالملفات بدلاً من رسالة خطأ، ابحث عن ملف نصي باسم يوحي بأنه الهدف (`flag.txt` مثلاً) واقرأه.

ما هي قيمة فلاج الـ Framework؟

أين تجد المعلومة؟ في أسفل الكود المصدري يوجد تعليق يذكر اسم الإطار البرمجي، إصداره، ورابط موقعه. افتح الرابط، وابحث في صفحة التوثيق أو سجل التحديثات (Changelog) عن أي إشارة لملف أو مسار يحتوي معلومات عن التحديث الأحدث مقارنة بالإصدار المستخدم في الموقع المستهدف.

استخرج الفلاجات الأربعة بنفسك باتباع الخطوات أعلاه على بيئتك الخاصة.

Inspector Tools: Developer — 4 Task

المفهوم

الفرق الجوهري بين **الكود المصدري (Page Source)** والـ **DOM الحي (Inspector)**: الأول هو ما استلمه المتصفح من الخادم فقط، أما الثاني فهو الحالة الحالية للصفحة بعد تنفيذ كل الـ JavaScript والـ CSS. الـ Inspector يتيح لك رؤية وتعديل هذه الحالة الحية — وهذا التعديل محلي فقط في متصفحك (لا يؤثر على الخادم أو المستخدمين الآخرين)، لكنه مفيد جداً لفهم كيف تُبنى آليات حجب المحتوى مثل الـ "Paywalls".

المنهجية

1. افتح قسم الأخبار (News)، ولاحظ أن أحد المقالات محجوب خلف طبقة علوية (Overlay/Paywall) تطلب اشتراكاً مدفوعاً.
2. كليك يمين على هذه الطبقة ← **Inspect**.
3. حدد العنصر (DIV) المسؤول عن الحجب في شجرة الـ HTML.
4. في لوحة الـ Styles، ابحث عن الخاصية `display`. القيمة `block` تعني "اعرض هذا العنصر"؛ غيّرها إلى `none` لإخفائه بالكامل من العرض، فيظهر المحتوى الذي كان خلفه.
5. طريقة بديلة: حدد العنصر في شجرة الـ DOM واضغط `Delete` لحذفه كلياً من الصفحة الحية.

لماذا يهم هذا من ناحية أمنية؟

هذا مثال تبسيطي على مبدأ مهم: **أي حماية تعتمد فقط على التحكم في العرض من جهة العميل (Client-Side) ليست حماية حقيقية.** إخفاء محتوى بـ CSS لا يمنع وصول بياناته الفعلية إلى المتصفح أصلاً — فهو موجود في الـ DOM، وبالتالي قابل للكشف بثوانٍ. الحماية الحقيقية يجب أن تكون من جهة الخادم (Server-Side): ألا يُرسل المحتوى المحجوب أصلاً للمستخدم غير المصرح له.

السؤال

ما هي القيمة المخفية خلف الـ Paywall؟

أين تجد المعلومة؟ بعد تنفيذ خطوات التعديل أعلاه على عنصر `premium-customer-blocker`، سيظهر النص المحجوب في الصفحة مباشرة.

استخرج القيمة بنفسك بتطبيق الخطوات.

Debugger Tools: Developer — 5 Task

المفهوم

تبويب الـ **Debugger** (ويسمى Sources في جوجل كروم) مخصص لتتبع وتصحيح كود الـ JavaScript. كمختبري اختراق، نستخدمه لفهم منطق كود JS معقد أو مموه (Ofbuscated)، والأهم: لإيقاف تنفيذ الكود عند نقطة معينة باستخدام **Breakpoints** لنرى حالة الصفحة قبل أن يُغيّرها الكود.

مفاهيم مهمة: - **Minfiication (التصغير)**: إزالة كل التنسيق (مسافات، أسطر جديدة) لتقليل حجم الملف — هذا لا يُعتبر حماية، فقط تحسين أداء. - **Ofbuscation (التمويه)**: تعقيد الكود عمداً لصعوبة قراءته من قبل البشر (أسماء متغيرات غير منطقية، بنية غير مباشرة). - **Print Pretty**: ميزة في أدوات المطور (أيقونة { }) تعيد تنسيق الكود المصغّر ليصبح مقروءاً أكثر (أسطر ومسافات)، لكنها لا تُلغي التمويه نفسه. - **Breakpoint**: نقطة توقف تجبر المتصفح على إيقاف تنفيذ الـ JavaScript عند وصوله لسطر معين، فتتجمد حالة الصفحة عند تلك اللحظة بالضبط.

المنهجية خطوة بخطوة

1. افتح صفحة Contact، ولاحظ ظهور ومضة حمراء سريعة عند تحميل الصفحة.
2. افتح الـ Debugger/Sources، وتصفح ملفات المشروع حتى تصل لملف JS مرتبط بهذه الظاهرة (عادة داخل مجلد assets، وله اسم يوحي بوظيفته مثل الإخفاء السريع).
3. اضغط Print Pretty { } لتنسيق الكود.
4. ابحث عن السطر الذي يستدعي دالة الإزالة (غالباً قرب نهاية الملف، ويحتوي كلمة تدل على الحذف/الإزالة).
5. اضغط على رقم السطر لوضع Breakpoint (يتحول لونه، عادة أزرق).
6. أعد تحميل الصفحة (F5). سيتوقف تنفيذ JS عند هذا السطر قبل أن يُنفَّذ، وبالتالي العنصر الذي كان سيُحذف سيبقى ظاهراً على الشاشة.

السؤال

ما هي القيمة الظاهرة داخل الصندوق الأحمر؟

أين تجد المعلومة؟ بعد إيقاف التنفيذ عبر الـ Breakpoint كما في الخطوات أعلاه، سيبقى الصندوق الأحمر ظاهراً على الصفحة وبداخله نص واضح.

استخرج القيمة بنفسك بتطبيق الخطوات.

Network Tools: Developer — 6 Task

المفهوم

تبويب الـ **Network** يسجل كل طلب شبكة HTTP) (Request تُصدره الصفحة — سواء عند التحميل الأولي، أو لاحقاً عبر **AJAX** (طلبات تُرسل وتُستقبل في الخلفية دون إعادة تحميل الصفحة بالكامل). هذا التبويب أساسي لأي اختبار اختراق لأنه يكشف لك:

- المسارات (Endpoints) الفعلية التي يتواصل معها تطبيق الـ Frontend خلف الكواليس.
- هيدرز الطلبات والاستجابات (قد تحتوي Tokens أو معلومات حساسة).
- محتوى الاستجابة الكامل Response) (Body حتى لو لم يُعرض شيء منه على الواجهة.

المنهجية

1. افتح صفحة ،Contact وافتح تبويب Network (استخدم أيقونة سلة المهملات لتفريغ القائمة إن احتجت بداية نظيفة).
2. املأ نموذج التواصل (اسم، بريد، رسالة) واضغط إرسال.
3. ستلاحظ طلب شبكة جديد يظهر في القائمة (هذا هو طلب الـ AJAX الذي أرسل بيانات النموذج في الخلفية).
4. اضغط على هذا الطلب، وانتقل إلى تبويب **Response** (أو Preview في بعض المتصفحات) لرؤية محتوى استجابة الخادم بالكامل.

لماذا يهم هذا؟

كثير من تطبيقات الويب الحديثة ،(SPA ،React (Vue تعتمد بشكل كامل على AJAX/Fetch/XHR للتواصل مع الـ Backend. مراقبة هذا التبويب غالباً ما تكشف Endpoints خاصة بالـ API لم تكن لتكتشفها بتصفح الصفحات العادية فقط.

السؤال

ما القيمة الموجودة داخل تبويب Response لطلب ؟contact-msg

أين تجد المعلومة؟ في تبويب Response للطلب الذي ظهر بعد إرسال نموذج التواصل، انظر لمحتوى الاستجابة الراجعة من الخادم نصياً.

استخرج القيمة بنفسك بتطبيق الخطوات.

Storage Tools: Developer — 7 Task

المفهوم

تبويب الـ **Storage** (أو Application في جوجل كروم) يعرض البيانات المخزنة من طرف العميل (المتصفح):

الوصف	النوع
بيانات تبقى محفوظة حتى بعد إغلاق المتصفح	Storage Local
بيانات مؤقتة لتبويب/جلسة واحدة فقط	Storage Session
بيانات صغيرة يرسلها الخادم ليحفظها المتصفح، تُستخدم غالباً للجلسات والمصادقة	Cookies
موارد مخزنة مؤقتاً (صور، سكربتات) لتسريع التحميل	Storage Cache

أهم شيء بالنسبة لمختبر الاختراق هو الـ Cookies، خصوصاً ثلاث خاصيات أمنية:

- **HttpOnly**: إذا كانت `true`، لا يستطيع الـ JavaScript قراءة هذه الكوكي (`document.cookie` لن يُظهرها). هذا يحمي من سرقة الكوكي عبر هجمات XSS. إذا كانت `false` أو غير مفعّلة، فالكوكي عرضة للسرقة في حال وجود ثغرة XSS في الموقع.
- **Secure**: تضمن أن الكوكي تُرسل فقط عبر HTTPS، وليس HTTP العادي.
- **SameSite**: يحدد متى تُرسل الكوكي مع طلبات من مواقع أخرى، ويساعد في التخفيف من هجمات CSRF.

المنهجية

1. أنشئ حساباً جديداً عبر `/customers/signup` وسجّل الدخول.
2. افتح تبويب Storage/Application ← Cookies ← اختر نطاق الموقع.
3. افحص عمود `HttpOnly` الخاص بكوكي الجلسة (Session Cookie).

السؤال

ما قيمة خاصية HttpOnly بعد تسجيل الدخول؟

أين تجد المعلومة؟ في جدول الكوكيز ضمن تبويب Storage، عمود HttpOnly المقابل لكوكي الجلسة.

لماذا يهم هذا؟ إذا كانت القيمة تدل على أن الحماية **غير مفعّلة**، فهذا يُعتبر نقطة ضعف تستحق التوثيق في أي تقرير اختبار اختراق حقيقي، لأنها تعني أن أي ثغرة XSS لاحقة في الموقع يمكن أن تُستغل مباشرة لسرقة جلسة المستخدم.

افحص الجدول بنفسك واستنتج القيمة.

Task 8 — Conclusion (خاتمة الروم)

لا يوجد سؤال تقني؛ هذا تاسك ختامي يلخص ما تم تعلمه. أهم خلاصة يجب أن ترسخ في ذهنك:

> **قبل أي أداة آلية، ابدأ دائماً بالفحص اليدوي.** التعليقات، الروابط المخفية، سلوك الـ JavaScript، طلبات الشبكة، وبيانات التخزين المحلي — كل هذه مصادر معلومات مجانية لا تحتاج أي استغلال فعلي، وغالباً ما تُغفلها الأدوات الآلية لأنها تعتمد على السياق والفهم البشري وليس فقط على مطابقة الأنماط (Pattern Matching).

الروم الثاني: Discovery Content

نظرة عامة على الروم

هذا الروم يُغطي **اكتشاف المحتوى** – أي العثور على صفحات وملفات ومجلدات لم تكن مقصودة لتكون متاحة للعامة أو غير مرتبطة بأي رابط ظاهر. هناك ثلاث مقاربات رئيسية تُغطيها الروم:

1. **الاكتشاف اليدوي (Discovery Manual)**: فحص ملفات تقليدية مثل `robots.txt` و `sitemap.xml`، وتحليل هيدرز HTTP وبصمة الـ Framework.
2. **OSINT (الاستخبارات مفتوحة المصدر)**: استغلال معلومات متاحة للعامة خارج الموقع نفسه – محركات البحث، أدوات تحديد التقنيات، أرشيف الإنترنت، GitHub، وحاويات S3.
3. **الاكتشاف الآلي (Discovery Automated)**: استخدام أدوات مثل Gobuster لتجربة آلاف المسارات بسرعة عبر قوائم كلمات (Wordlists).

Introduction — 1 Task (الإعداد)

تاسك إعدادي فقط، تشغيل الآلات والتأكد من الوصول للموقع المستهدف. لا سؤال تقني.

Files Common Discovery: Manual — 2 Task (الملفات الشائعة)

المفهوم

ملف robots.txt

هذا الملف موجّه لمحركات البحث (Crawlers/Bots) ويخبرها بما يُسمح وما لا يُسمح بأرشفته. **مهم جداً أن تفهم أنه توجيه وليس تحكماً أمنياً** – أي مسار مذكور تحت `Disallow:` لا يعني أنه محمي فعلياً، بل فقط أن محركات البحث "الملتزمة" (مثل جوجل) لن تُدرجه في نتائج البحث. أي شخص يمكنه زيارة هذا المسار مباشرة طالما يعرف عنوانه – وهذا بالضبط ما يجعله مصدر معلومات ذهبي لمختبر الاختراق: المطورون أحياناً يضعون هنا مسارات حساسة ظنّاً منهم أنها "مخفية".

ملف sitemap.xml

على عكس `robots.txt`، هذا الملف يسرد الصفحات التي **يريد** صاحب الموقع أرشفتها من قبل محركات البحث. أحياناً يحتوي صفحات تجريبية، قديمة، أو مسارات يصعب الوصول إليها بالتصفح العادي.

المنهجية

افتح `http://MACHINE_IP/robots.txt` و `http://MACHINE_IP/sitemap.xml` مباشرة في المتصفح أو عبر `curl`، واقرأ محتواهما بعناية.

الأسئلة

ما هو المجلد الموجود في robots.txt والممنوع من الزحف ؟(Disallow)

أين تجد المعلومة؟ في محتوى ملف `robots.txt`، ابحث عن السطر الذي يبدأ بـ `Disallow:`.

ما مسار المنطقة السرية الموجودة في sitemap.xml؟

أين تجد المعلومة؟ تصفح قائمة المسارات المذكورة في `sitemap.xml`، وابحث عن مساراً لا يتبع التسمية المنطقية للصفحات العادية (اسم غريب أو مكتوب بطريقة ملتوية/leetspeak).

استنتج القيمتين بنفسك من محتوى الملفين.

Task 3 — Framework & Headers Discovery: Manual (الهيدرز وبصمة الإطار)

المفهوم

هيدرز استجابة HTTP تكشف معلومات تقنية مهمة: - `Server`: نوع وإصدار خادم الويب. - `X-Powered-By`: اللغة/الإطار المستخدم في التطبيق. - أي هيدر مخصص (Custom Header) قد يضعه المطورون لأغراض تصحيح الأخطاء (Debugging) وينسون إزالته قبل النشر.

معرفة الـ Framework المستخدم تفتح الباب لزيارة موقعه الرسمي والبحث في توثيقه عن: البنية الافتراضية للمجلدات، مسار لوحة التحكم الإدارية الافتراضي، وأحياناً بيانات اعتماد افتراضية (Default Credentials) لم يتم تغييرها.

المنهجية

1. نفّذ `curl -v http://MACHINE_IP` وتفحص **كل** الهيدرز المُرجعة بعناية، ليس فقط `Server` و `X-Powered-By` — أي هيدر غير معتاد يستحق الانتباه.
2. اعرض الكود المصدري وابحث عن تعليق يذكر اسم الإطار ورابط موقعه الرسمي في أسفل الصفحة.
3. افتح رابط موقع الإطار، وتصفح قسم التوثيق (Documentation) بحثاً عن مسار لوحة التحكم الإدارية الافتراضي.
4. جرّب الدخول للوحة التحكم ببيانات اعتماد افتراضية شائعة (مثل admin/admin — هذا نمط هجوم حقيقي يُستخدم في اختبارات الاختراق عندما يُكتشف إطار عمل مخصص أو أقل شهرة لم يُغيّر المطورون بياناته الافتراضية.

الأسئلة

ما قيمة الهيدر المخصص X-FLAG؟

أين تجد المعلومة؟ نفّذ `curl -v http://MACHINE_IP` أو `curl -I http://MACHINE_IP` وتفحص كامل قائمة الهيدرز الراجعة في الاستجابة.

ما القيمة الموجودة في لوحة تحكم الإطار الإدارية؟

أين تجد المعلومة؟ بعد الوصول لرابط الإطار من التعليق أسفل الصفحة، ابحث في توثيقه عن مسار صفحة تسجيل الدخول الإدارية، وجرّب بيانات الدخول الافتراضية الشائعة. بعد الدخول الناجح ستظهر رسالة ترحيب تحتوي القيمة المطلوبة.

استخرج القيمتين بنفسك بتطبيق الخطوات أعلاه.

Task 4 — OSINT: Search Engines & Web Tools (محركات البحث والأدوات)

المفهوم

Google Dorking (أو Google Hacking)

استخدام مُعاملات بحث متقدمة في جوجل لتضييق النتائج وكشف محتوى مُفهرس لم يكن يُفترض أن يكون عاماً. من أهم هذه المعاملات:

المعامل	الوظيفة
`site:`	فقط معين نطاق على النتائج حصر
`inurl:`	نفسه الرابط ضمن كلمة عن البحث
`filetype:`	معين ملف نوع على النتائج حصر (PDF، XLS، إلخ)
`intitle:`	الصفحة عنوان ضمن كلمة عن البحث
`intext:`	الصفحة نص محتوى ضمن كلمة عن البحث
`cache:`	الصفحة من جوجل لدى المخزنة النسخة عرض

يمكن دمج أكثر من معامل في استعلام واحد لتضييق النتائج بدقة أكبر (مثال: البحث عن ملفات PDF مفهرسة من نطاق معين فقط).

Wappalyzer

إضافة متصفح وأداة ويب تُحلل استجابات الموقع، ملفات الـ JS، والهيدرز، لتكشف تلقائياً التقنيات المستخدمة: نظام إدارة المحتوى (CMS)، الإطار البرمجي، خادم الويب، أدوات التحليل (Analytics)، شبكات توصيل المحتوى (CDN)، وأحياناً أرقام الإصدارات.

الأسئلة

ما هو معامل Dork Google الذي يحصر النتائج على موقع معين؟

أين تجد المعلومة؟ راجع جدول المعاملات أعلاه، وحدد المعامل المرتبط بتحديد نطاق/موقع البحث تحديداً.

ما اسم الأداة (وإضافة المتصفح) التي تحدد التقنيات المستخدمة في موقع؟

أين تجد المعلومة؟ هذا مذكور صراحة في شرح التاسك كاسم أداة مستقلة ذُكرت بالتفصيل.

استنتج الإجابتين من النص أعلاه مباشرة؛ هذا السؤال معرفي بحت.

Task 5 — OSINT: Archives & Repositories (الأرشيف والمستودعات)

المفهوم

Wayback Machine (أرشيف الإنترنت)

خدمة تحفظ نسخاً تاريخية (Snapshots) من صفحات الويب منذ أواخر التسعينيات. مفيدة جداً لإيجاد: - صفحات حُذفت من الموقع الحالي لكن قد تبقى تعمل فعلياً على الخادم. - نماذج تسجيل دخول قديمة. - نقاط API منسية لم تعد مرتبطة من الواجهة الحالية لكنها لا تزال نشطة.

GitHub

منصة استضافة مستودعات Git. المطورون أحياناً يرفعون بيانات حساسة (مفاتيح API، كلمات مرور، ملفات env.) بالخطأ قبل أن يدركوا أن المستودع عام (Public). **نقطة مهمة جداً:** حتى لو حُذفت هذه البيانات في Commit لاحق، فإنها **تبقى موجودة في سجل التاريخ (Commit History)** ويمكن استرجاعها، لأن Git يحتفظ بكل التغييرات السابقة ما لم يُعاد كتابة التاريخ (History Rewrite) بشكل متعمد.

Buckets S3 (حاويات أمازون التخزينية)

خدمة تخزين سحابي شائعة الاستخدام لاستضافة الملفات والمواقع الثابتة. الصيغة القياسية للرابط: `https://{name}.s3.amazonaws.com`.

الخطر هنا هو **سوء ضبط الصلاحيات** — إذا تُركت الحاوية عامة بالخطأ (Publicly Accessible)، يصبح محتواها متاحاً للجميع. أنماط تسمية شائعة تستحق التجربة: `{company}-assets`، `{company}-backup`، `{company}-www`، `{company}-dev`.

الأسئلة

ما هو عنوان موقع Wayback Machine؟

أين تجد المعلومة؟ مذكور صراحة في شرح التاسك كعنوان URL رئيسي لخدمة الأرشيف.

ما الصيغة التي تنتهي بها روابط حاويات S3؟ (تبدأ الإجابة بنقطة .)

أين تجد المعلومة؟ راجع صيغة الرابط القياسية الموضحة في شرح Buckets S3 أعلاه، وحدد الجزء الثابت الذي يأتي بعد اسم الحاوية.

استنتج الإجابتين من الشرح أعلاه مباشرة.

Task 6 — Gobuster Discovery: Automated Fundamentals (أساسيات Gobuster)

المفهوم

عندما تنفد الطرق اليدوية والـ OSINT، يأتي دور **الفحص الآلي**: إرسال مئات أو آلاف الطلبات بسرعة لاختبار وجود مسارات معينة، اعتماداً على **قوائم كلمات (Wordlists)** تحتوي أسماء ملفات ومجلدات شائعة.

Gobuster

أداة مفتوحة المصدر مكتوبة بلغة Go، تدعم عدة أوضاع (Modes): `dir` - تخمين الملفات والمجلدات على خادم ويب. `dns` - تخمين النطاقات الفرعية (Subdomains). `vhost` - تخمين الـ Virtual Hosts.

أهم الرايات (Flags) العامة

الوظيفة	الراية
عدد الخيوط المتوازية (أسرع = ضغط أكبر على الخادم)	`--threads` / `-t`
مسار قائمة الكلمات (إلزامية)	`--wordlist` / `-w`
حفظ النتائج في ملف	`--output` / `-o`
وقت انتظار بين الطلبات (مفيد ضد خوادم محدودة المعدل)	`--delay`

قوائم الكلمات (Wordlists)

SecLists هي المجموعة الأشهر والأكثر استخداماً، مثبتة مسبقاً على الـ AttackBox في `/usr/share/wordlists/SecLists/`.
لفحص المجلدات، الملفات الأكثر استخداماً: `Discovery/Web-Content/common.txt` و `Discovery/Web-Content/directory-list-2.3-medium.txt`.

وضع dir

الصيغة الأساسية:

```
gobuster dir -u "http://MACHINE_IP" -w /path/to/wordlist
```

الرايتان u- و w- إلزاميتان دائماً. رايات إضافية مفيدة:

الراية	الوظيفة
-x	امتدادات ملفات للبحث عنها (مثل: .php,.txt,.js)
-r	التوجيه إعادة متابعة (Redirects)
-k	تجاهل التحقق من شهادة TLS
-s	فقط محددة حالة أكواد عرض

كيف تقرأ مخرجات Gobuster؟

كل سطر في النتائج يمثل مساراً تم العثور عليه، مع كود الحالة (Status Code) وحجم الاستجابة (Size)، وفي حال وجود إعادة توجيه، المسار الذي يُوجَّه إليه. ركّز على: - أكواد 200: المسار موجود ومتاح مباشرة. - أكواد 301/302: إعادة توجيه (قد يكشف مسارات إضافية). - أسماء ملفات تحمل امتداد .log، .bak، .old، أو أي اسم يوحي بأنه غير مخصص للنشر العام.

الأسئلة

ما اسم المجلد الذي يبدأ بـ mo/ والذي تم اكتشافه؟

أين تجد المعلومة؟ نفّذ فحص Gobuster بوضع dir ضد الهدف باستخدام قائمة common.txt، وراجع المخرجات بحثاً عن مسار يبدأ بالأحرف المحددة.

ما اسم ملف السجلات (Log) الذي تم اكتشافه؟

أين تجد المعلومة؟ في نفس مخرجات الفحص، ابحث عن مسار له امتداد يدل على كونه ملف سجلات.

نفّذ الفحص بنفسك واستنتج الاسمين من المخرجات.

Task 7 — Subdomains & Virtual Hosts Discovery: Automated (النطاقات الفرعية والمضيفون الافتراضيون)

المفهوم

الفرق بين Subdomain و Host Virtual — هذا هو أهم مفهوم في هذا التاسك

المفهوم	كيف يُحل	مثال
Subdomain	عبر DNS — سجل يُترجم اسم النطاق إلى عنوان IP	`blog.example.thm` له سجل DNS خاص به
(vhost) Host Virtual	عبر خادم الويب نفسه، باستخدام هيدر `Host:` في طلب HTTP لتحديد أي موقع يجب خدمته	عدة مواقع تعمل على نفس الـ IP، والخادم يفرّق بينها بقراءة هيدر `Host`

بعبارة أخرى: الـ Subdomain يُحلّ قبل وصول الطلب للخادم (في مرحلة DNS)، بينما الـ Host Virtual يُحلّ بعد وصول الطلب للخادم (الخادم نفسه يقرر أي تطبيق يخدم الطلب بناءً على الهيدر). هذا يعني أن فحص الـ vhost لا يعتمد على DNS إطلاقاً، بل يرسل طلبات HTTP مباشرة لنفس الـ IP مع تغيير قيمة هيدر `Host` في كل مرة.

لماذا هذا مهم أمنياً؟ موقع فرعي أو Host Virtual قد يحتوي ثغرة غير موجودة في النطاق الرئيسي — ربما نسخة قديمة، بيئة تطوير أو اختبار منسية، أو تطبيق مختلف كلياً يُستضاف على نفس الخادم.

تجهيز البيئة

قبل البدء، يجب تعديل إعدادات الـ DNS محلياً على الـ AttackBox لضمان حل النطاقات التجريبية المستخدمة في الروم بشكل صحيح:

1. عدّل `/etc/resolv-dnsmasq` وأضف `nameserver MACHINE_IP` كأول سطر.
2. أعد تشغيل خدمة dnsmasq.
3. أضف تعيين النطاق المستهدف إلى عنوان الـ IP في ملف `/etc/hosts`.

وضع dns

يقوم بعمليات بحث DNS فعلية باستخدام كل كلمة في قائمة الكلمات كنطاق فرعي محتمل. الرايات الإلزامية: `-d` (النطاق) و `-w` (قائمة الكلمات). الراية `--wildcard` تجبر الفحص على الاستمرار حتى لو اكتُشف وجود Wildcard DNS (حيث يستجيب أي نطاق فرعي عشوائي، مما قد يُنتج نتائج إيجابية كاذبة).

```
gobuster dns -d example.thm -w /path/to/wordlist --wildcard
```

وضع vhost

لا يعتمد على DNS إطلاقاً — يرسل طلبات HTTP مباشرة إلى عنوان الـ IP المستهدف، مع تغيير قيمة هيدر `Host:` في كل طلب لتجربة كل كلمة من القائمة كمضيف افتراضي محتمل.

```
gobuster vhost -u "http://MACHINE_IP" --domain example.thm -w /path/to/wordlist --append-domain --exclud
```

- `--append-domain`: يدمج كل كلمة من القائمة مع النطاق الأساسي (بدلاً من استخدام الكلمة وحدها).
- `--exclude-length`: يستبعد النتائج ذات حجم استجابة معين — مفيد جداً لتصفية النتائج الإيجابية الكاذبة (False Positives) التي تشترك في نفس حجم استجابة الصفحة الافتراضية/الخطأ.

الأسئلة

بجانب dns و w-، ما هي الراية المختصرة الإلزامية الأخرى في وضع dns؟

أين تجد المعلومة؟ راجع الصيغة الأساسية لوضع dns أعلاه، وحدد الراية الإلزامية الثالثة بجانب تحديد نوع الفحص وقائمة الكلمات — وهي المسؤولة عن تحديد النطاق المستهدف نفسه.

كم عدد الـ Hosts Virtual التي ترجع كود حالة 200 على acmeitsupport.thm؟

أين تجد المعلومة؟ أضف النطاق المطلوب إلى /etc/hosts مرتبطاً بعنوان الـ IP الخاص بالآلة، ثم نفّذ فحص vhost كما في الصيغة أعلاه مستبدلاً domain-- بالنطاق الصحيح. عُدّ عدد الأسطر في المخرجات النهائية التي تحمل كود حالة 200 تحديداً (وليس أي كود آخر).

نفّذ الفحصين بنفسك واستنتج الإجابتين من المخرجات الفعلية.

Task 8 — Conclusion (خاتمة الروم)

لا يوجد سؤال تقني. الخلاصة الأهم من هذا الروم:

منهجية اكتشاف المحتوى الكاملة تجمع بين ثلاث طبقات متكاملة وليست بديلة عن بعضها: الفحص اليدوي (سريع ويكشف أشياء محددة بدقة)، الـ OSINT (يستغل معلومات متاحة للعامة خارج نطاق سيطرة الموقع نفسه)، والفحص الآلي (يغطي اتساعاً لا يمكن تحقيقه يدوياً). في أي اختبار اختراق حقيقي، شغّل الثلاثة قبل الانتقال لمرحلة الاستغلال.

الروم الثالث: Stacks Web Modern

نظرة عامة على الروم

هذا الروم يركز على **بصمة الـ Stack (Fingerprinting)** كمُضاعِف لسرعة الاستغلال: معرفة الإطار البرمجي بالضبط (وإصداره) تعني معرفة الثغرة الدقيقة المطبّقة بدلاً من إهدار وقت في فحص عشوائي. المنهجية المتبعة في كل تاسك ثابتة:

1. **بصمة الـ Stack** من إشارات HTTP السلبية (بدون إرسال أي Payload استغلالي).
2. **تأكيد الإصدار** وتحديد الثغرة (CVE) المطابقة.
3. **تنفيذ سلسلة الاستغلال** وفهم السبب الجذري للثغرة.

الآلة الهدف تُشغّل أربع خدمات على أربعة منافذ مختلفة: 3000 ،(MERN/Express) 3001 ،(Next.js) 8000 ،(Django) 8080 (Apache 2.4.49).

Task 1 — Introduction (الإعداد)

تاسك إعدادي، تشغيل الآلة والتأكد من جاهزية المنافذ الأربعة. لا سؤال تقني.

Task 2 — MERN Stack (Express) + Prototype Pollution

المفهوم: بصمة Express

الإشارة	القيمة	الثقة مستوى
X-Powered-By هيدر	`Express`	عالٍ
الكوكي اسم	`connect.sid=s%3A...`	عالٍ
موجود غير مسار استجابة	`Cannot GET /path` عادي نص	عالٍ

ملاحظة مهمة: هيدر `X-Powered-By: Express` يُرسل افتراضياً من Express، لكنه يُحذف تلقائياً إذا استدعى المطور `app.disable('x-powered-by')` أو استخدم مكتبة Helmet، كما أن بعض الخوادم الوسيطة (Reverse Proxy) أو منصات الاستضافة قد تُزيله قبل وصوله للعميل. كوكي `connect.sid` يأتي من حزمة `express-session`، وقد لا يظهر إن كان الإعداد `saveUninitialized: false` (وهذا شائع في تطبيقات تسجيل الدخول الحديثة) — غيابه لا يعني غياب Express، فقط استخدم إشارات أخرى كتأكيد بديل.

المفهوم: Pollution Prototype (تلويث الـ Prototype)

هذا مفهوم جوهري في أمن JavaScript/Node.js يستحق فهماً عميقاً وليس حفظاً سطحياً.

في JavaScript، كل كائن (Object) يرث خصائصه من كائن جذر مشترك يُسمى `Object.prototype`. عندما تكتب دالة "دمج" (Merge) تقوم بنسخ خصائص من كائن المصدر (Source) إلى كائن الهدف (Target) **دون تصفية أسماء المفاتيح (Keys)**، يصبح بالإمكان تمرير مفتاح خاص اسمه `__proto__` ضمن بيانات JSON المُرسلة. عندما تصل الدالة لهذا المفتاح، فإنها لا تتعامل معه كخاصية

عادية، بل كمرجع مباشر إلى `Object.prototype` نفسه — وبالتالي أي قيمة تُكتب هناك **تصبح متاحة لكل كائن آخر في نفس عملية Node.js** يبحث عن نفس اسم الخاصية عبر سلسلة الوراثة (Prototype Chain)، حتى لو لم تُحدَّد هذه الخاصية صراحة على ذلك الكائن.

هذا بالضبط ما يحدث في ثغرة CVE-2020-8203 (كما هو مذكور في جدول الخلاصة بنهاية الروم): إذا كان هناك مسار إداري يتحقق من خاصية مثل `currentUser.isAdmin`، وتمكّنا من تلويث `Object.prototype.isAdmin = true`، فإن **أي** كائن `currentUser` — حتى لو لم يُحدَّد له صراحة `isAdmin` من قبل — سيُرجع `true` عند قراءة هذه الخاصية، لأن محرك JavaScript يبحث عبر سلسلة الوراثة تلقائياً.

تحليل الكود الضعيف

```
function merge(target, source) {
  for (let key in source) {
    if (typeof source[key] === 'object' && source[key] !== null) {
      if (!target[key]) target[key] = {};
      merge(target[key], source[key]);
    } else {
      target[key] = source[key];
    }
  }
  return target;
}
```

الخلل هنا هو غياب أي تصفية (Filtering/Sanitization) لأسماء المفاتيح الخطيرة مثل `__proto__` أو `constructor.prototype` قبل الاستخدام في حلقة `for...in`. **ملاحظة متقدمة:** بعض التطبيقات المُحصّنة تُصفّي `__proto__` تحديداً عند مستوى الإدخال، لكن مسار `constructor.prototype` يصل لنفس الهدف (`Object.prototype`) من طريق مختلف، وقد يتجاوز هذا النوع من التصفية الجزئية.

المنهجية خطوة بخطوة

1. **التحقق من البصمة:** نفّذ `curl -I MACHINE_IP:3000/` وافحص الهيدرز.
2. **تأكيد أن المسار الإداري محمي فعلياً:** احفظ كوكي الجلسة، ثم جرّب الوصول لمسار الفلاج الإداري دون أي تعديل — يجب أن تحصل على رفض (Not Authorized).
3. **فهم سلوك نقطة التحديث العادية:** جرّب إرسال بيانات JSON طبيعية (مثل تغيير الاسم) لمسار التحديث، ولاحظ أن الخادم يقبل أي مفتاح JSON دون تصفية — هذا هو مؤشر الثغرة.
4. **إرسال حمولة التلويث:** أرسل طلب POST لنفس نقطة التحديث، لكن هذه المرة بمفتاح `__proto__` يحتوي الخاصية الإدارية المطلوب تفعيلها بقيمة `true`.
5. **التحقق من نجاح التلويث:** أعد طلب المسار الإداري بنفس كوكي الجلسة — إن نجحت الثغرة، سيُرجع الخادم محتوى الفلاج لأن فحص الصلاحية أصبح يُرجع `true` عبر سلسلة الوراثة الملوَّثة.

الأسئلة

ما الهيدر الذي يؤكد تشغيل خادم Express.js؟ (الصيغة: Header-Name: Value)

أين تجد المعلومة؟ في مخرجات `curl -I` ضد المنفذ 3000، ابحث عن الهيدر الذي يُصدره Express تلقائياً ما لم يُعطَّل صراحة.

ما اسم كوكي الجلسة الخاص بـ Express الذي ستستخدمه لإعادة إرسال الطلبات بعد تلويث الـ Prototype؟

أين تجد المعلومة؟ في نفس مخرجات الهيدرز، ابحث عن هيدر `Set-Cookie` واسم الكوكي المرتبط بحزمة `express-session`.

أرسل حمولة تلويث الـ Prototype لنقطة الدمج. ما القيمة التي يُرجعها المسار الإداري بعد نجاح التجاوز؟

أين تجد المعلومة؟ بعد تنفيذ خطوات الاستغلال أعلاه بالكامل (حفظ الكوكي ← إرسال حمولة التلويث ← طلب المسار الإداري بنفس الكوكي)، ستظهر القيمة في استجابة JSON النهائية.

نفّذ سلسلة الاستغلال كاملة بنفسك واستنتج القيم الثلاث من مخرجاتك الفعلية.

Task 3 — React / Next.js (Middleware Bypass)

المفهوم: بصمة Next.js

مستوى الثقة	القيمة	الإشارة
عالٍ	`Next.js`	هيدر X-Powered-By
عالٍ جداً (يؤكد Router App تحديداً)	وجود `window.__next_f` داخل وسم script	الكود المصدري
عالٍ	`/_next/static/chunks/`	مسارات الأصول الثابتة
متوسط	`x-middleware-next` أو `x-middleware-rewrite`	هيدرز الـ Middleware
متوسط	كود 307 نحو صفحة تسجيل الدخول	إعادة توجيه لمسار محمي

المفهوم: Server Components وبروتوكول Flight

في نمط Router App (منذ Next.js 13، افتراضي منذ 14)، تُنفَّذ مكونات React **على الخادم مباشرة** بدلاً من إرسال كود JS للمتصفح لتنفيذها هناك، ثم تُبَثّ النتيجة للعميل بصيغة خاصة تُسمى **RSC Flight Protocol**. هذا التطور المعماري هو بالضبط ما فتح الباب أمام فئة جديدة من الثغرات غير الموجودة في الإصدارات الأقدم من Next.js.

المفهوم: CVE-2025-29927 — تجاوز الـ Middleware

الـ **Middleware** في Next.js هي دالة تُنفَّذ قبل وصول أي طلب لصفحته المستهدفة. المطورون يستخدمونها غالباً كبوابة مركزية واحدة للتحقق من المصادقة (Authentication) والجلسات — وهذا بالضبط ما يجعل اختراقها خطيراً جداً: إذا تجاوزتها، فأنت تتجاوز **كل** آليات الحماية المبنية عليها دفعة واحدة.

آلية الثغرة: يستخدم Next.js داخلياً هيدراً باسم `x-middleware-subrequest` لمنع حلقات تكرار لا نهائية (عندما تستدعي الـ Middleware نفسها لإعادة توجيه الطلب). المشكلة الجوهرية: **لم يتحقق Next.js أبداً من كون هذا الهيدر قادماً من عملية داخلية فعلية أو من طلب خارجي عادي من أي عميل.** ببساطة، إذا أضفت هذا الهيدر بنفسك في طلبك، يُعامله Next.js وكأنه طلب داخلي شرعي ويتخطى تنفيذ الـ Middleware بالكامل — بما في ذلك فحص المصادقة.

قيمة الهيدر تتكوّن من مسار وحدة الـ Middleware مكرراً خمس مرات متتالية مفصولة بنقطتين (:). إن كان ملف `middleware.ts` في جذر المشروع، فالقيمة تختلف عمّا لو كان داخل مجلد `src/` — يجب التحقق من بنية المشروع المستهدف أولاً إن أمكن، أو تجربة الاحتمالين.

هذه الثغرة مصنّفة CVSS 9.1 (حرجة): تجاوز مصادقة كامل بهيدر HTTP واحد فقط، دون أي بيانات اعتماد أو جلسة.

CVE-2025-55182 (مرجع فقط دون تطبيق في هذا الروم)

ثغرة تنفيذ أوامر عن بُعد (RCE) غير مصادق عليها في محلل بروتوكول RSC، Flight عبر إلغاء تسلسل (Deserialization) غير آمن. مُصنّفة CVSS 10.0 وتصيب نطاقاً واسعاً من إصدارات Next.js 14/15. التفصيل الكامل خارج نطاق هذا الروم ومُغطّى في روم مخصص منفصل.

المنهجية خطوة بخطوة

1. **التحقق من البصمة**: نفّذ `curl -I http://MACHINE_IP:3001/` وابحث عن هيدر `X-Powered-By` وهيدرز `x-nextjs-*`.
2. **البحث عن مؤشر Router App**: اعرض الكود المصدري وابحث عن `window.__next_f`.
3. **تأكيد أن المسار المحمي يعمل فعلاً**: جرّب طلب المسار المحمي (Dashboard) بدون أي هيدر إضافي — يجب أن تُعاد توجيهك لصفحة تسجيل الدخول.
4. **تنفيذ التجاوز**: أرسل نفس الطلب، لكن أضف هيدر `x-middleware-subrequest` بالقيمة الصحيحة (اسم ملف الـ Middleware مكرراً خمس مرات بفواصل :).

الأسئلة

ما العنصر الموجود في الكود المصدري الذي يؤكد أن التطبيق يستخدم Next.js App Router؟

أين تجد المعلومة؟ اعرض الكود المصدري لأي صفحة في التطبيق، وابحث عن متغير JavaScript مُحقن تلقائياً يبدأ بـ `window.__next_f` — وهو خاص حصرياً بنمط Router App.

أرسل هيدر تجاوز CVE-2025-29927 لمسار dashboard/ المحمي. ما القيمة التي تظهر على الصفحة؟

أين تجد المعلومة؟ نفّذ طلب `curl` يتضمن هيدر `x-middleware-subrequest` بقيمة اسم ملف الـ Middleware مكرراً خمس مرات (تحقق أولاً هل هو في الجذر أم داخل `src/`)، موجّهاً إلى مسار `/dashboard`. القيمة ستظهر مباشرة في محتوى الاستجابة.

نفّذ الفحص والتجاوز بنفسك، واستنتج القيمتين من مخرجاتك الفعلية.

Task 4 — Django (SQL Injection) عبر order_by

المفهوم: بصمة Django

مستوى الثقة	القيمة	الإشارة
عالٍ	`CPython/X.X.X WSGIServer/0.2`	هيدر Server
عالٍ	`csrftoken`	اسم الكوكي
عالٍ	`DENY`	هيدر X-Frame-Options

مستوى الثقة	القيمة	الإشارة
عالٍ	`nosniff`	هيدر X-Content-Type-Options
متوسّط	`same-origin`	هيدر Referrer-Policy
عالٍ جداً — شبه قطعي	`csrfmiddlewaretoken`	حقل مخفي في أي فورم POST

أهم بصمة هنا هي حقل `csrfmiddlewaretoken` المخفي، لأن `CsrfViewMiddleware` في Django يُدرجه تلقائياً في كل فورم POST — ولن تجده أبداً في Express أو Rails أو Next.js. كذلك، اجتماع ثلاثة هيدرز أمنية معاً (`X-Frame-Options: DENY` + `X-Content-Type-Options: nosniff` + `Referrer-Policy: same-origin`) هو بصمة قوية لـ `SecurityMiddleware` الخاص بـ Django تحديداً.

المفهوم: CVE-2021-35042 — حقن SQL عبر `order_by()`

الـ **ORM** (Object-Relational Mapper) في Django مصمم أصلاً ليحمي المطورين من حقن SQL تلقائياً، لأنه يبني الاستعلامات بطريقة آمنة بمعاملات مُجهَّزة (Parameterized). لكن هذه الحماية **تنهار تماماً** عندما يتجاوز المطور الـ ORM ويبني استعلام SQL يدوياً عبر دمج/تجميع نصوص (String Concatenation) تحتوي مدخلات مستخدم مباشرة.

تحليل الكود الضعيف

```
order = self.request.GET.get('order', 'name')
sql = (
    'SELECT id, name, price, description FROM products_product '
    f'ORDER BY (CASE WHEN (1=1) THEN {order} ELSE name END)')
```

القيمة القادمة من بارامتر `order` في رابط الصفحة تُدرج **مباشرة وحرفياً** داخل جملة SQL دون أي تصفية أو معاملات مُجهَّزة. بنية `CASE WHEN (1=1) THEN ... ELSE ...` دائماً تُنفّذ فرع `THEN` لأن الشرط `1=1` صحيح دوماً — وهذا بالضبط ما يجعله نقطة الحقن الفعلية.

تقنية الاستخراج: Error-Based updatexml() Injection

دالة `updatexml()` في MySQL تُستخدم أصلاً لتعديل مستندات XML، لكنها تُطلق خطأً إذا كانت صيغة الـ XPath المُمررة لها غير صالحة. التقنية هنا تستغل هذا السلوك: بتضمين استعلام SELECT فرعي داخل تعبير XPath باستخدام `concat()`، فإن رسالة الخطأ الناتجة (غير الصالحة أصلاً بسبب XPath خاطئ عمداً) **تتضمن نتيجة الاستعلام الفرعي ضمن نص رسالة الخطأ نفسها**. القيمة السداسية عشرية `0x7e` تقابل الرمز ~ وتُستخدم كفاصل يسهّل تمييز بداية القيمة المُستخرجة داخل رسالة الخطأ.

شرط أساسي لنجاح هذه التقنية: يجب أن يكون وضع التصحيح `DEBUG = True` مفعّلاً في إعدادات Django، لأن هذا هو ما يجعل رسائل خطأ MySQL الكاملة تظهر في استجابة HTTP 500 بدلاً من صفحة خطأ عامة. **في اختبار اختراق حقيقي، تحقق من هذا الشرط أولاً** — فإن كان `DEBUG = False`، التقنية البديلة هي الحقن الأعمى المعتمد على الوقت (Time-Based Blind) باستخدام `SLEEP()`.

المنهجية خطوة بخطوة

1. **التحقق من البصمة:** نفّذ `curl -I` ضد المنفذ 8000، وابحث عن هيدر الخادم وكوكي `csrftoken`.

2. **تحديد نقطة الحقن**: اعرض صفحة /products/، ولاحظ وجود بارامتر order في الفورم يتحكم فيه المستخدم مباشرة.
3. **استخراج إصدار قاعدة البيانات**: أرسل حمولة updatexml() تستعلم عن @@version عبر بارامتر order، وابحث في الاستجابة عن النص المحدد بالفاصل ~.
4. **استخراج اسم قاعدة البيانات**: كرر نفس الأسلوب لكن استبدل الاستعلام الفرعي بـ database().

الأسئلة

ما الحقل المخفي في نماذج POST الخاصة بـ Django والذي يُعتبر بصمة شبه قطعية؟

أين تجد المعلومة؟ اعرض الكود المصدري لأي فورم POST في التطبيق، وابحث عن حقل <input type="hidden"> باسم محدد يُدرجه Django تلقائياً.

باستخدام حمولات curl يدوية، ما اسم قاعدة البيانات الضعيفة؟

أين تجد المعلومة؟ نفّذ طلب curl يستخدم تقنية updatexml() الموضحة أعلاه، مستبدلاً الاستعلام الفرعي بدالة تُرجع اسم قاعدة البيانات الحالية المتصل بها التطبيق. استخرج القيمة من رسالة الخطأ المحددة بالفاصل ~.

نفّذ الحقن بنفسك واستنتج القيمة من رسالة الخطأ الفعلية.

Task 5 — Apache LAMP (2.4.49 Traversal Path RCE)

المفهوم: بصمة Apache / LAMP

أبسط بصمة ممكنة: هيدر Server يُظهر رقم الإصدار صراحة (Apache/2.4.49 (Unix))**.** هذا الإصدار بالذات يُطابق ثغرة واحدة محددة تماماً: **CVE-2021-41773.** كذلك، تذييل صفحات الخطأ الافتراضية (404، 403) في Apache يُكرر رقم الإصدار، فحتى لو حاول أحدهم إخفاء هيدر Server، فقد ينسى تعديل صفحات الخطأ الافتراضية.

مؤشر إضافي مهم: الاستجابة عند طلب /cgi-bin/ — إن كانت 403 Forbidden، فهذا يعني أن المجلد **موجود** لكن الفهرسة (Listing) معطّلة، وهو ما يدل على أن mod_cgi مُفعّل (شرط أساسي لاستغلال هذه الثغرة)**.** أما 404 فتعني أن المجلد غير موجود أصلاً**.**

المفهوم: CVE-2021-41773

أدخل إصدار Apache 2.4.49 تعديلاً على دالة داخلية اسمها ap_normalize_path()، وهذا التعديل كسر بالخطأ آلية تصفية مسارات الـ Traversal (/../)**.** السبب الجذري هو **خلل في ترتيب عمليات فك الترميز (Decode) (Order)**: فلتر منع الـ Traversal يعمل قبل فك الترميز الكامل للرابط URL) ،(Decoding وليس بعده**.**

عندما ترسل تسلسلاً مثل /%2e. (نقطة حرفية، متبوعة بنقطة مُرمّزة بصيغة ،URL متبوعة بشرطة مائلة)، فإن فلتر Apache لا يتعرف على هذا كمعادل لـ /../ لأنه لم يُفك ترميزه بعد عند لحظة الفحص. لكن عندما يُمرَّر الرابط لاحقاً إلى نظام الملفات، يُفكّ نظام التشغيل الترميز فعلياً، ويُفسِّر /%2e. على أنه /../ حرفياً — وبذلك تم تجاوز الفلتر.

وحدها، هذه ثغرة قراءة ملفات عبر Path .Traversal لكن **الخطورة الحقيقية** تأتي من التفاعل مع mod_cgi: مجلد /cgi-bin/ لديه صلاحية تنفيذ CGI مفعّلة**.** فإذا انتهى مسار الـ Traversal إلى ملف قابل للتنفيذ مثل /bin/sh، فإن Apache يُشغّله **كسكربت CGI**،

ويُمرّر محتوى جسم طلب POST مباشرة كمدخل قياسي (stdin) لهذا السكربت — وهذا يعني تنفيذ أوامر عن بُعد كاملاً .(RCE)

لماذا راية path-as-is– ضرورية؟

أداة curl تُطبّع (Normalize) الروابط افتراضياً قبل إرسالها — أي أنها "تُنظّف" تسلسلات مثل /%2e. بنفسها محلياً قبل أن يغادر الطلب جهازك، فيصل الخادم مساراً عادياً لا أثر فيه لمحاولة الالتفاف. الراية path-as-is-- توقف هذا السلوك، وتضمن إرسال الرابط تماماً كما كتبته حرفياً.

متطلبات صيغة CGI

بروتوكول CGI يتطلب كتلة هيدرز HTTP صالحة قبل جسم الاستجابة، يليها سطر فارغ. لهذا، أي حمولة استغلال يجب أن تبدأ بـ echo قبل الأمر الفعلي المطلوب تنفيذه — وإلا سيرفض Apache الطلب بخطأ 500. echo; text/plain; Content-Type:

المنهجية خطوة بخطوة

1. **تأكيد البصمة**: نفّذ curl -I ضد المنفذ 8080، وتحقق من قيمة هيدر Server.
2. **تأكيد تفعيل mod_cgi**: اطلب /cgi-bin/ وتحقق من حصولك على 403 (موجود) وليس 404 (غير موجود).
3. **تأكيد تنفيذ الأوامر عن بُعد**: أرسل طلب curl بالراية path-as-is--، مع مسار Traversal يصعد من /cgi-bin/ إلى /bin/sh (أربع تكرارات لـ /%2e.)، مع تمرير أمر بسيط مثل id في جسم الطلب مسبوقاً بمقدمة CGI الصحيحة.
4. **توسيع الاستغلال**: بعد تأكيد التنفيذ، استبدل الأمر بأي أمر نظام آخر (قراءة ملفات، استعراض حسابات النظام، إلخ).

الأسئلة

ما القيمة الدقيقة لهيدر Server التي تحدد الهدف كضعيف أمام CVE-2021-41773؟ (الصيغة: Apache/X.X.XX (OS))

أين تجد المعلومة؟ في مخرجات curl -I ضد المنفذ 8080، قيمة هيدر Server كاملة بما فيها اسم نظام التشغيل بين قوسين.

ما راية curl الضرورية لمنعه من تطبيع تسلسلات الـ Traversal في الرابط قبل الإرسال؟

أين تجد المعلومة؟ هذه الراية مذكورة صراحة في شرح التاسك ومستخدمة في كل أوامر الاستغلال الموضحة هناك.

ما محتوى ملف flag.txt؟

أين تجد المعلومة؟ نفّذ أمر curl باستخدام تقنية الـ Traversal Path المذكورة أعلاه (الصعود من /cgi-bin/ إلى /bin/sh بأربع تكرارات /%2e.)، مع تمرير أمر قراءة الملف (/flag.txt cat) مسبوقاً بمقدمة CGI الصحيحة ضمن جسم الطلب.

نفّذ سلسلة الاستغلال كاملة بنفسك واستنتج القيم الثلاث من مخرجاتك الفعلية.

Task 6 — Automation (الفحص الآلي بـ Nikto)

المفهوم

بعد إتقان الفحص اليدوي، يأتي دور الأتمتة لتسريع الاستطلاع الأولي عبر عدة أهداف في وقت واحد. **Nikto** أداة لفحص خوادم ويب تقرأ الهيدرز والسلوكيات وتقارنها بقاعدة بيانات توقيعات (Signatures) معروفة لكشف أخطاء إعداد وبرمجيات قديمة بسرعة،

لكنها **لا تملك** القدرة على اكتشاف ثغرات منطقية معقدة تتطلب فهم سياق التطبيق (مثل Pollution Prototype أو حقن SQL في استعلامات ORM مخصصة) — تلك تحتاج دائماً تحليلاً يدوياً كما فعلنا في التاسكات 2 و4.

المنهجية

نفّذ Nikto ضد كل منفذ من المنافذ الأربعة على حدة (`http://MACHINE_IP:PORT -h nikto`)، وقارن ما تُخرجه الأداة تلقائياً مع ما توصلت إليه يدوياً في التاسكات السابقة. لاحظ كيف أن Nikto يستطيع فوراً استخراج رقم إصدار Apache الدقيق (لأنه بصمة بسيطة من هيدر مباشر)، لكنه لا يستطيع اكتشاف ثغرة حقن SQL الموجودة في بارامتر `order` على Django، لأن هذا يتطلب فهماً للمنطق البرمجي وليس فقط مطابقة أنماط الاستجابة.

السؤال

هذا تاسك تطبيقي للتدرب على الأتمتة — تأكيد فقط بدون قيمة محددة.

لا يوجد سؤال تقني يتطلب استخراج قيمة؛ فقط نفّذ الفحوصات الأربعة بنفسك وقارن مخرجاتها بما توصلت إليه يدوياً.

Task 7 — Conclusion (خاتمة الروم)

لا يوجد سؤال تقني. جدول ملخص الثغرات الأربعة المُغطاة في هذا الروم للمراجعة:

الأثر	الثغرة (CVE)	الـ Stack
تلويث Prototype ← تجاوز المصادقة	CVE-2020-8203	Express / MERN
هيدر واحد ← تجاوز كامل لـ Middleware	CVE-2025-29927	Middleware Next.js
حقن SQL عبر order_by غير مُعامَل	CVE-2021-35042	ORM Django
mod_cgi + Traversal Path ← تنفيذ أوامر عن بُعد	CVE-2021-41773	LAMP Apache

الدرس الأهم من هذا الروم: كل Stack يترك بصمته الخاصة، وبمجرد أن تتقن قراءة هذه الإشارات، تتحول من التخمين العشوائي إلى الاستهداف الدقيق المبني على أدلة — وهذا بالضبط الفرق بين فحص آلي صاخب (Noisy Scanner) وهجوم موجّه مبني على الفهم.

الروم الرابع: I - Attacks Server Web

نظرة عامة على الروم

هذا الروم يركّز على **مرحلة الاستطلاع وتحديد أخطاء الإعداد (Misconfiguration) (Identfiication** عبر أربعة خوادم ويب مختلفة تمثل الأنواع السائدة في البنية التحتية المبنية على :Linux

- **Apache2**: الخادم التقليدي الأكثر انتشاراً.
- **server HTTP built-in Python's**: الخادم "العرضي" الذي يظهر كثيراً أكثر مما يُتوقع.
- **Node.js (Express)**: نمط خادم التطبيقات الحديث.
- **Nginx**: خادم ويب وProxy عكسي عالي الأداء.

ملاحظة مهمة: هذا الروم يتوقف عند **تحديد** أخطاء الإعداد فقط — لا يوجد استغلال فعلي يؤدي لـ Shell أو RCE أو تصعيد صلاحيات هنا؛ هذا هو موضوع الروم التالي (Web Attacks Server - II الخاص بـ .(IIS

Introduction — 1 Task (الإعداد)

تاسك إعدادي فقط. انتظر حوالي دقيقتين حتى تكتمل جاهزية الخدمات الأربع على منافذها.

Servers Web Identfiying — 2 Task (تحديد هوية الخوادم)

المفهوم

معرفة نوع خادم الويب تحدد **أي أخطاء إعداد محتملة أصلاً**، وأي أدوات ستكون الأكثر فاعلية. أغلب الخوادم "تُعرّف عن نفسها" من خلال:

1. **هيدر Server**: الإشارة الأكثر مباشرة.
2. **هيدر X-Powered-By**: خاص بأطر التطبيقات (مثل Express التي لا تُرسل هيدر Server تقليدي.
3. **صفحات الخطأ الافتراضية (404)**: شكل مختلف تماماً لكل خادم، ومفيد حتى لو أُخفي هيدر .Server

جدول الهيدرز الافتراضية للمنافذ الأربعة (كما هي في بيئة هذا الروم)

المنفذ	الخادم	هيدر Server الافتراضي
80	Apache2	`Apache/2.4.x (Ubuntu)`
8000	Python HTTP Server	`SimpleHTTP/0.6 Python/3.xx.x`
3000	Node.js Express	(يعتمد يوجد لا على `X-Powered-By`)
8080	Nginx	`nginx/1.xx.x`

المنهجية

```
# □□□ HEAD □□□ □□□□□□) □□□□ □□□ (□□□□□□□□□□
curl -sI http://MACHINE_IP:PORT
```

```
# □□□ GET □□□□ □□□□□ □□□□ □□□□□ □□□□□□□□□□ □□□ □□□□ □□□ □□□□□
curl -s http://MACHINE_IP:PORT/nonexistent-page-xyz
```

نفّذ هذين الأمرين ضد كل منفذ من الأربعة، ولاحظ الفروقات في شكل صفحة الخطأ 404 بين كل خادم — Python تُرجع نصاً عادياً بسيطاً، Nginx يذكر إصداره في تذييل صفحة HTML، وApache يذكر اسمه في جسم الصفحة.

الأسئلة

ما قيمة هيدر Server لخادم Server HTTP Python على المنفذ 8000؟ (الصيغة: SimpleHTTP/X.X Python/X.X.X)

أين تجد المعلومة؟ نفّذ `curl -sI` ضد المنفذ 8000 مباشرة، واقرأ قيمة هيدر `Server` بالكامل.

أي هيدر HTTP يكشف إطار التطبيق العامل على المنفذ 3000؟

أين تجد المعلومة؟ نفّذ `curl -sI` ضد المنفذ 3000. بما أن Express لا يُرسل هيدر `Server` تقليدياً، ابحث عن الهيدر البديل المخصص لكشف الإطار البرمجي.

ما خادم الويب العامل على المنفذ 8080؟

أين تجد المعلومة؟ نفّذ `curl -sI` ضد المنفذ 8080 واقرأ قيمة هيدر `Server`.

نفّذ الأوامر الثلاثة بنفسك واستنتج القيم من المخرجات الفعلية.

Task 3 — Server HTTP Python (لا يوجد تحكم وصول إطلاقاً)

المفهوم

خادم Python المدمج (`python3 -m http.server`) أداة رائعة للمطورين لمشاركة ملفات بسرعة، لكنه **لا يحتوي أي آلية تحكم وصول** — لا مصادقة، لا قوائم حظر، لا ملف إعدادات شبيه بـ `.htaccess`. إذا كان الملف موجوداً فعلياً في المجلد الذي يخدمه، فهو متاح لأي شخص يستطيع الوصول للمنفذ، **بما في ذلك الملفات المخفية التي تبدأ بنقطة (Dotfiles) مثل `.env`** — وهذا فرق جوهري عن خوادم مثل Apache أو Nginx التي تُخفي Dotfiles تلقائياً أو يمكن ضبطها لذلك.

السيناريو الواقعي: مطور يُشغّل هذا الخادم "مؤقتاً" لمشاركة ملف أو اختبار شيء سريع، ثم ينسى إيقافه — فيبقى يعمل لأشهر على خادم متاح للعامة أو شبكة داخلية، وكل ما بداخل مجلد العمل مكشوف بالكامل.

المنهجية

1. **استعراض فهرس المجلد:** إذا لم يوجد ملف `index.html`، يُولّد الخادم تلقائياً صفحة HTML تسرد كل الملفات.
2. **الوصول للملفات المخفية مباشرة:** جرّب طلب `.env` مباشرة حتى لو لم يظهر في قائمة الفهرسة (بعض الإعدادات تُخفيه من العرض لكن لا تمنع الوصول المباشر).
3. **تحميل وفحص أي أرشيف مضغوط:** إذا وجدت ملف `.zip` أو `.tar.gz`، حمّله وافحص محتواه — المطورون أحياناً يتركون نسخاً احتياطية تحتوي كود مصدري أو تفريغات قواعد بيانات (Database Dumps).

```
curl -s http://MACHINE_IP:8000/.env
curl -s http://MACHINE_IP:8000/backup.zip -o backup.zip
unzip backup.zip -d backup-contents/
```

الأسئلة

ما كلمة مرور قاعدة البيانات المخزنة في ملف env. الذي يخدمه خادم Python؟

أين تجد المعلومة؟ اطلب env. مباشرة عبر curl، وابحث عن سطر الاتصال بقاعدة البيانات (غالباً بصيغة DATABASE_URL=...)، وحدد كلمة المرور الموجودة بين اسم المستخدم والمضيف.

ما القيمة الموجودة داخل الأرشيف المضغوط الذي يخدمه خادم Python؟

أين تجد المعلومة؟ حمّل الأرشيف المذكور في قائمة الفهرسة، فك ضغطه، وابحث داخل الملفات الناتجة (خصوصاً أي تفريغ قاعدة بيانات أو ملاحظات نصية) عن القيمة المطلوبة.

نفّذ الأوامر بنفسك واستنتج القيمتين من المحتوى الفعلي الذي تحصل عليه.

Task 4 — Apache2، mod_status) Directory، Listing والملفات المنسية)

المفهوم

كشف الإصدار

curl -sI - يكشف هيدر Server مباشرة. على أوبونتو، الإعداد الافتراضي ServerTokens OS يُضيف اسم نظام التشغيل بجانب رقم الإصدار.

Listing Directory

توجيه Options +Indexes في إعدادات Apache يجعل الخادم يعرض قائمة بمحتويات أي مجلد لا يحتوي ملف فهرس افتراضي (index.html). قد يكون هذا مقصوداً (مشاركة ملفات داخلية) أو خطأ غير مقصود على مسار يحتوي بيانات حساسة.

صفحة mod_status

وحدة مدمجة في Apache تعرض إحصائيات حية عن حالة الخادم عبر مسار /server-status. **الإعداد الصحيح** يقتصر الوصول على localhost فقط عبر توجيه local Require. **المشكلة الشائعة**: وجود توجيه granted all Require في أي مكان ضمن إعدادات الـ Host Virtual قد **يتجاوز بصمت** هذا التقييد الافتراضي، فيُصبح المسار مكشوفاً لأي عنوان IP — حتى لو بدا أن باقي الإعدادات "افتراضية". هذه الصفحة تكشف: الاتصالات النشطة والمسارات المطلوبة، إجمالي عدد الطلبات، حالات العمال (Workers)، وإصدار الخادم ووقت تشغيله.

اكتشاف الملفات غير المرتبطة عبر Gobuster

ليس كل شيء مرتبط برابط ظاهر أو موجود في قائمة فهرسة. ملفات النسخ الاحتياطي والإعدادات القديمة والملفات التجريبية المنسية غالباً ما تكون موجودة في جذر المستندات دون أي رابط إليها — ويُكتشف وجودها فقط بتخمين أسمائها عبر قوائم كلمات.

```
gobuster dir -u http://TARGET_IP:80 -w /usr/share/wordlists/SecLists/Discovery/Web-Content/common.txt -x
```

عند العثور على ملف بامتداد `bak.`، يستحق التحميل والفحص الفوري دائماً — غالباً ما يحتوي أجزاء إعدادات أو بيانات اعتماد. كذلك ملفات `htpasswd`. (قائمة مستخدمين وكلمات مرور مُشفَّرة لمصادقة HTTP الأساسية) تستحق اهتماماً خاصاً إن ظهرت — تؤكد استخدام Auth Basic في جزء من الموقع، وتتيح محاولة كسر التشفير (Cracking) دون اتصال.

المنهجية خطوة بخطوة

1. تحقق من هيدر الإصدار.
2. تصفح أي مجلد يسمح بالفهرسة (مثل `/files/`) واقرأ كل ملف بداخله.
3. تحقق من `server-status/` مباشرة حتى لو بدت باقي الإعدادات افتراضية.
4. شغّل Gobuster بامتدادات `bak,txt` على الأقل لاكتشاف ملفات منسية غير مرتبطة.

الأسئلة

ما وحدة Apache المسؤولة عن كشف إحصائيات الخادم الحية عند server-status/؟

أين تجد المعلومة؟ هذا مفهوم أساسي مذكور صراحة في شرح التاسك — اسم الوحدة البرمجية المسؤولة عن توليد هذه الصفحة بالتحديد.

ما القيمة الموجودة في مجلد /files/ على Apache؟

أين تجد المعلومة؟ تصفح `/files/` مباشرة (يجب أن تظهر فهرسة للمجلد)، واقرأ محتوى الملف أو الملفات الموجودة بداخله.

نفّذ الفحوصات بنفسك واستنتج الإجابتين من مخرجاتك الفعلية.

Task 5 — Node.js (Express): التسريبات في وضع التطوير

المفهوم

تطبيقات Node.js لا تخدم ملفات ثابتة من مجلد إعدادات بسيط — إنها **تنفّذ كوداً برمجياً** يقرر ماذا يُعاد لكل طلب. هذه المرونة قوية، لكنها مصدر أخطاء شائعة: مطورون يتركون ميزات وضع التطوير (Development Mode) مُفعّلة في بيئة الإنتاج.

أنماط التسريب الشائعة

1. **رسائل خطأ مفصّلة (Verbose Errors)**: معالج الأخطاء المدمج في Express يُخفي تتبعات الأخطاء Stack Traces تلقائياً عندما يكون `NODE_ENV=production`. لكن **المطورين غالباً يكتبون معالجات أخطاء مخصصة خاصة بهم**، وهذه المعالجات قد تُظهر التتبع الكامل بغض النظر عن قيمة `NODE_ENV` — لأن السلوك الافتراضي لـ Express لا يُطبَّق أصلاً إذا تم تجاوزه بكود مخصص. أي استجابة `Error Server Internal 500` تستحق فحصاً دقيقاً لجسم الاستجابة.
2. **نقاط Debug لسرد المسارات (Routes)**: بعض المطورين يبنون مساراً مخصصاً يُسرد كل مسارات التطبيق المسجّلة (لتسهيل التطوير)، وينسون حذفه قبل النشر. هذا يوفر وقتاً هائلاً في الاستطلاع مقارنة بتخمين المسارات عبر Gobuster.
3. **متغيرات البيئة المكشوفة**: نقطة Debug ترجع محتوى `process.env` كاملاً قد تكشف كلمات مرور قواعد البيانات، مفاتيح API، وقيمة `NODE_ENV` نفسها (التي إن كانت `development` على خادم إنتاج، فهذا مؤشر واضح على نشر غير مُحصَّن).

4. **ملفات ثابتة تحتوي معلومات داخلية**: ملفات JS تُخدم للعميل (عبر `express.static()`) أحياناً تحتوي ثوابت مثل روابط API داخلية، أسماء مضيفين داخليين، أو أعلام تصحيح Debug Flags).

ملاحظة تقنية مهمة: ميزة `express.static()` تُرجع `404` تلقائياً لأي ملف يبدأ بنقطة (Dotfile) — وهذا **عكس تماماً** سلوك خادم Python (الذي يخدم كل شيء بلا استثناء). فإذا حاولت الوصول لملف `.env` عبر مسار ثابت في Express وحصلت على 404، فهذا لا يعني غياب الملف، بل أن الـ Middleware يحجبه تحديداً.

المنهجية خطوة بخطوة

1. **تأكيد الإطار**: `curl -sI` للتحقق من `X-Powered-By: Express`.
2. **قراءة نسخة التطبيق**: المسار الجذري غالباً يُرجع استجابة JSON تحتوي حالة التطبيق وإصداره.
3. **محاولة إثارة خطأ**: جرّب الوصول لمسار API قد يفشل (مثل مسار يعتمد على اتصال قاعدة بيانات)، وافحص جسم استجابة الخطأ 500 كاملاً.
4. **البحث عن نقطة سرد المسارات**: جرّب مسارات شائعة مثل `/api/routes` لسرد كل نقاط النهاية المسجّلة في التطبيق.
5. **البحث عن نقطة متغيرات البيئة**: جرّب مسارات شائعة مثل `/api/debug/env`.
6. **افحص الملفات الثابتة**: بعد معرفة المسارات من نقطة السرد، افحص أي ملف JS يُخدم كملف ثابت.

الأسئلة

ما قيمة متغير NODE_ENV في استجابة نقطة الـ debug؟

أين تجد المعلومة؟ اطلب نقطة متغيرات البيئة (`/api/debug/env` أو ما يعادلها) وابحث عن مفتاح `NODE_ENV` في استجابة الـ JSON.

ما القيمة الموجودة في الملفات الثابتة التي يخدمها تطبيق Node.js؟

أين تجد المعلومة؟ بعد معرفة مسار الملفات الثابتة من نقطة سرد المسارات، اطلب الملف المرتبط (غالباً ملف إعدادات JS للواجهة الأمامية) وافحص محتواه بالكامل.

نفّذ الفحوصات بنفسك واستنتج القيمتين من مخرجاتك الفعلية.

Task 6 — Nginx (autoindex) و (stub_status)

المفهوم

نفس فئات الأخطاء المتكررة (كشف الإصدار، فهرسة المجلدات، صفحة حالة مكشوفة) لكن بمفردات إعداد مختلفة خاصة بـ Nginx.

كشف الإصدار

توجيه `server_tokens` يتحكم في ظهور رقم الإصدار، سواء في هيدر `Server` أو في تذييل صفحات الخطأ الافتراضية **في آن واحد.** القيمة الافتراضية `on` تعني كشف الإصدار. **نقطة تحقق مهمة**: إذا بدا أن هيدر `Server` مُخفى، تحقق من صفحة خطأ 404 الافتراضية — إن ظهر رقم الإصدار هناك، فهذا يعني أن `server_tokens` لم يُضبط فعلياً على `off` بشكل كامل.

توجيه autoindex

يُفعّل فهرسة المجلدات صراحة (على عكس Apache الذي لا يحتاج توجيهاً صريحاً بنفس الطريقة):

```
location /files/ {
    autoindex on;
    root /var/www/nginx/;
}
```

هذا توجيه موثّق ومُستخدم شرعياً في حالات مشاركة ملفات داخلية — المشكلة هي استخدامه على مسار يحتوي بيانات حساسة، أو تركه مفعّلاً دون أي قيود وصول إضافية.

وحدة stub_status ومسار nginx_status

تعرض مقاييس اتصال حية. **الإعداد الآمن**: `allow 127.0.0.1; deny all;`. **المشكلة الشائعة**: استبدال هذا بـ `allow all;` يفتح الوصول للجميع. شكل الاستجابة مضغوط وبدون تسميات واضحة للأرقام:

```
Active connections: 1
server accepts handled requests
 15 15 15
Reading: 0 Writing: 1 Waiting: 0
```

السطر الثاني من الأرقام الثلاثة يمثل بالترتيب: إجمالي الاتصالات المقبولة، إجمالي الاتصالات المُعالجة، إجمالي الطلبات منذ بدء تشغيل الخادم.

المنهجية خطوة بخطوة

1. تحقق من هيدر `Server`، وإن بدا مخفياً، تحقق من صفحة الخطأ 404 كتأكيد إضافي.
2. تصفح أي مسار يُحتمل تفعيل `autoindex` عليه (مثل `/files/`) واقرأ كل ملف بداخله.
3. تحقق من `/nginx_status` مباشرة.

الأسئلة

ما التوجيه في Nginx الذي يُفعّل فهرسة المجلدات داخل location block؟

أين تجد المعلومة؟ مذكور صراحة في شرح التاسك كاسم التوجيه المسؤول عن هذه الميزة.

ما مسار الرابط الذي يكشف إحصائيات اتصال Nginx؟

أين تجد المعلومة؟ مذكور صراحة في شرح التاسك كاسم المسار الافتراضي المرتبط بوحدة `stub_status`.

ما القيمة الموجودة في فهرسة المجلد على المنفذ 8080؟

أين تجد المعلومة؟ تصفح المسار الذي يظهر فيه `autoindex` مفعّلاً، واقرأ محتوى الملفات النصية الموجودة بداخله (وإن وُجد أرشيف مضغوط، حمّله وافحص محتواه أيضاً).

نفّذ الفحوصات بنفسك واستنتج القيم الثلاث من مخرجاتك الفعلية.

Task 7 — Common Misconfigurations Across Servers (الأنماط المشتركة بين كل الخوادم)

المفهوم: هيدرز الأمان المفقودة

هيدرز استجابة HTTP تُوجّه سلوك المتصفح لحمايته من فئات هجوم محددة من جهة العميل:

الهيدر	يحمي من	قيمة مثال
`X-Frame-Options`	Clickjacking (منع تضمين الصفحة داخل firame من نطاق آخر)	`DENY` أو `SAMEORIGIN`
`X-Content-Type-Options`	Snffiing MIME (منع تخمين المتصفح لنوع المحتوى)	`nosniff`
`Content-Security-Policy`	يُقيّد من أين تُحمَّل السكربتات والموارد	`default-src 'self'`
`Referrer-Policy`	يتحكم فيما يُرسل ضمن هيدر Referer عند الانتقال لصفحة أخرى	`no-referrer` أو `strict-origin`
`Strict-Transport-Security`	يفرض HTTPS للطلبات اللاحقة (ذو معنى فقط عبر HTTPS)	`max-age=31536000`

ملاحظة فنية: هيدر `X-Frame-Options` بات تقنياً مُستبدلاً بتوجيه `frame-ancestors` ضمن سياسة `Content-Security-Policy` الذي يوفر تحكماً أدق. المواقع الحديثة المُحصّنة جيداً قد تعتمد على CSP وحده فتُغفل `X-Frame-Options` عمداً — لذا عند التوثيق، تحقق من الاثنين معاً قبل اعتبار الحماية غائبة.

المنهجية

```
for port in 80 8000 3000 8080; do
  echo "=== Port $port ==="
  curl -sI http://MACHINE_IP:$port/ | grep -iE "x-frame-options|x-content-type|content-security-policy|s
done
```

المفهوم: الفحص الآلي بـ Nikto

أداة فحص سريعة وصاخبة (تُولّد حركة مرور كبيرة ويسهل كشفها) — مناسبة للاختبارات المصرّح بها رسمياً، وليست أداة تخفٍّ. الراية `-nointeractive` تمنع الأداة من انتظار أي تدخل يدوي أثناء الفحص. راية `-Tuning` تتيح تقييد أنواع الفحوصات المُنفّذة لتسريع العملية (الأكواد تُكتب متتالية بدون فواصل).

الأنماط المشتركة (جدول ملخص)

النمط	Apache	HTTP Python	Node.js	Nginx
كشف الإصدار في الهيدرز	نعم	نعم	جزئي	نعم
فهرسة المجلدات	على مسارات محددة	المسار الجذري	غير منطبق	على مسارات محددة
نقطة حالة/تصحيح مكشوفة	`/server-status`	غير منطبق	نقاط `/api/debug/...`	`/nginx_status`

النمط	Apache	HTTP Python	Node.js	Nginx
ملفات حساسة متاحة	ملفات `.bak` ونسخ احتياطية	`.env`، أرشيفات	ملفات JS للإعدادات	ملفات نصية للنشر
هيدرز أمان مفقودة	نعم	نعم	نعم	نعم

الخلاصة الأهم: الإعدادات الافتراضية في كل هذه الخوادم **تُفضّل سهولة النشر على الأمان.** كشف الإصدار وفهرسة المجلدات وصفحات الحالة مُفعّلة افتراضياً لأغراض تشخيصية تُسهّل عمل المسؤولين — وتعطيلها أو تقييدها يتطلب إجراءً واعياً ومتعمداً. وجود هذه الأنماط في اختبار اختراق حقيقي لا يعني بالضرورة إهمالاً جسيماً، بل غالباً يعني أن الإعدادات الافتراضية لم تُراجع أصلاً منذ النشر.

الأسئلة

أي هيدر أمان، عند غيابه، يسمح بتضمين الصفحة داخل firame من نطاق آخر؟

أين تجد المعلومة؟ راجع جدول هيدرز الأمان أعلاه، وحدد الهيدر المرتبط تحديداً بحماية Clickjacking.

ما نص النتيجة التي يُصدرها Nikto عند اكتشافه فهرسة مفعّلة على مسار /files/؟

أين تجد المعلومة؟ نفّذ `nikto -h http://MACHINE_IP:80 -nointeractive` وابحث في المخرجات عن السطر الذي يبدأ بمعرّف OSVDB ويذكر مسار `/files/` واكتشاف الفهرسة تحديداً.

نفّذ الفحص بنفسك واستنتج الإجابتين من الشرح والمخرجات الفعلية.

Task 8 — Conclusion (خاتمة الروم)

لا يوجد سؤال تقني. أهم النقاط للمراجعة الذهنية:

هيدرز `Server` و `X-Powered-By` هي أسرع سطح لبصمة أي تطبيق — تكشف نوع البرمجية وأحياناً الإصدار قبل حتى إرسال طلب واحد لمسار تطبيقي فعلي.

خادم Python المدمج لا يملك أي تحكم وصول إطلاقاً — افترض دائماً أن كامل مجلد العمل قابل للقراءة، بما فيه الملفات المخفية.

وحدة `mod_status` في Apache وفهرسة المجلدات مُفعّلتان افتراضياً على نُسخ أوبونتو، وتتطلبان إعداداً نشطاً لتقييدهما — وجودهما شائع ويستحق الفحص الدائم.

تطبيقات Node.js/Express في وضع التطوير تُسرّب تتبعات الأخطاء، قوائم المسارات، ومتغيرات البيئة — كل واحدة منها تُعتبر نتيجة (Finding) مستقلة بذاتها، ومجتمعة تعطي صورة كاملة عن البنية الداخلية للتطبيق.

توجيه `autoindex` ووحدة `stub_status` في Nginx يُعادلان نمطي فهرسة المجلدات وصفحة الحالة في Apache — المفردات مختلفة، لكن منهجية الاستطلاع واحدة تماماً.

هيدرز الأمان غائبة افتراضياً على الخوادم الأربعة جميعاً — فحصها عبر `curl -sI` مع `grep` على أسماء الهيدرز فحص سريع وموثوق يجب أن يكون جزءاً من كل اختبار اختراق لخادم ويب.

ملخص سريع للمراجعة النهائية (Cheat Sheet)

الروم 1 — Application An Walking

- الكود المصدري الـ DOM الحي: الأول ثابت من الخادم، الثاني حي وقابل للتعديل المؤقت عبر الـ Inspector.
- Breakpoints في الـ Debugger توقف تنفيذ JS قبل سطر معين — مفيدة لمنع كود يُخفي/يحذف محتوى من التنفيذ.
- تبويب Network يكشف طلبات AJAX الخفية ومسارات API غير الظاهرة في التصفح العادي.
- خاصية HttpOnly على الكوكيز تمنع وصول JavaScript لها — غيابها يعني قابلية السرقة عبر XSS.

الروم 2 — Discovery Content

- `robots.txt` توجيه لمحركات البحث فقط، وليس تحكماً أمنياً.
- `sitemap.xml` قد يكشف مسارات مقصودة للأرشفة لكنها غير ظاهرة في التصفح العادي.
- حقل `csrfmiddlewaretoken`-مثل بصمة الأطر غالباً موجود في فورمات POST وقد يكشف الإطار المستخدم.
- Subdomain يُحل عبر DNS؛ Host Virtual يُحل عبر هيدر `Host` في خادم الويب نفسه — الفرق جوهري لفهم أوضاع `dns` و`vhost` في Gobuster.

الروم 3 — Stacks Web Modern

- `X-Powered-By: Express` + كوكي `connect.sid` = بصمة MERN قوية.
- Pollution: Prototype تلويث `__proto__` يصل لكل كائن JS في نفس العملية عبر سلسلة الوراثة.
- هيدر `x-middleware-subrequest` في Next.js كان يُعامَل كطلب داخلي موثوق دون تحقق (CVE-2025-29927).
- حقل `csrfmiddlewaretoken` = بصمة Django شبه قطعية؛ خطر الـ ORM يظهر فقط عند تجاوزه ببناء SQL يدوي.
- `%2e/` في Apache 2.4.49 يتجاوز فلتر Traversal بسبب ترتيب خاطئ لفك الترميز (CVE-2021-41773).

الروم 4 — Attacks Server Web - I

- خادم HTTP Python لا يحجب أي شيء، حتى الـ Dotfiles مثل `.env`.
- `/server-status` (Apache) و `/nginx_status` (Nginx) صفحات حالة حية قد تُكشف بالخطأ عبر `Require all granted` أو `allow all`.
- Express في وضع التطوير يُسرّب Stack Traces، قوائم المسارات، ومتغيرات البيئة عبر نقاط Debug منسية.
- هيدرز الأمان (`X-Frame-Options`, `CSP`, إلخ) غائبة افتراضياً على كل الخوادم الأربعة ما لم تُضبط يدوياً.

قائمة مراجعة المهارات

بعد إتمام هذا المديول، يجب أن تكون قادراً على شرح كل ما يلي من الذاكرة، دون الرجوع لهذا الدليل:

- ☐ الفرق بين الكود المصدري الثابت والـ DOM الحي، ومتى تستخدم كل منهما.
- ☐ كيفية استخدام Breakpoints لإيقاف تنفيذ JavaScript عند نقطة محددة.
- ☐ كيفية تتبع طلبات AJAX عبر تبويب Network واستخراج معلومات من استجاباتها.
- ☐ دلالة خاصيات الكوكيز الأمنية الثلاث: HttpOnly، Secure، SameSite.
- ☐ الفرق الوظيفي بين `robots.txt` و `sitemap.xml`.
- ☐ أهم معاملات Dorking Google وكيفية دمجها.
- ☐ لماذا سجل الـ History Commit في GitHub لا يزال خطراً حتى بعد حذف البيانات الحساسة في commit لاحق.
- ☐ الفرق الجوهري بين Subdomain (DNS) و Host Virtual (هيدر Host في الخادم).
- ☐ آلية Pollution Prototype وكيف يصل `__proto__` إلى `Object.prototype`.
- ☐ كيف يستغل CVE-2025-29927 غياب التحقق من مصدر هيدر `x-middleware-subrequest`.
- ☐ كيف تعمل تقنية `updatexml()` في استخراج بيانات عبر رسائل خطأ MySQL، ولماذا تتطلب `DEBUG = True`.
- ☐ كيف يتجاوز `/%2e`. فلتر الـ Traversal Path في Apache 2.4.49 بسبب ترتيب فك الترميز.
- ☐ لماذا خادم HTTP Python المدمج يُعتبر خطراً كاملاً بمجرد وجوده في النطاق، دون أي استغلال فعلي.
- ☐ الفرق بين `mod_status` (Apache) و `stub_status` (Nginx) من ناحية الوظيفة والمخاطر.
- ☐ لماذا `NODE_ENV=development` على خادم إنتاج يُعتبر علامة تحذير قائمة بذاتها.
- ☐ القائمة الكاملة لهيدرز الأمان الخمسة وما يحمي كل واحد منها.